Estatística Bayesiana

Jose Storopoli
josees@uni9.pro.br Universidade Nove de Julho - UNINOVE

Sumário

1. Estatística Bayesiana
2. Distribuições Probabilísticas
3. `rstanarm` e `brms`
4. Priors
5. Markov Chain Monte Carlo - MCMC
6. Regressão Linear Bayesiana
7. Regressão Logística Bayesiana
8. Regressão de Poisson Bayesiana
9. Regressão Robusta Bayesiana
10. Modelos Multiníveis Bayesianos
11. Comparação de Modelos
12. Referências

Sumário para Estatística Bayesiana

1.1 Leituras Recomendadas
1.2 O que é Estatística Bayesiana
1.2.1 O que muda da Estatística Frequentista?
1.3 Ferramentas
1.3.1 Stan
1.3.2 R
1.3.3 Python
1.3.4 Julia
1.3.5 Turing
1.4 Probabilidade
1.4.1 PROBABILIDADE NÃO EXISTE!
1.4.2 O que é Probabilidade?
1.4.3 Probabilidade Condicional
1.4.4 Probabilidade Conjunta
1.4.5 Teorema de Bayes
1.4.6 Espaços Amostrais

Estatística Bayesiana - Leituras Recomendadas

- Gelman et al. (2013) - Capítulo 1: Probability and inference
- McElreath (2020) - Capítulo 1: The Golem of Prague
- Gelman et al. (2020) - Capítulo 3: Some basic methods in mathematics and probability
- Storopoli (2021b) - O que é Estatística Bayesiana?

O que é Estatística Bayesiana

A estatística Bayesiana[1] é uma abordagem de análise de dados baseada no teorema de Bayes, onde o conhecimento disponível sobre os parâmetros em um modelo estatístico é atualizado com as informações dos dados observados (Gelman et al., 2013). O conhecimento prévio é expresso como uma distribuição *a priori*[2] e combinado com os dados observados na forma de uma função de verossimilhança[3] para determinar a distribuição *a posteriori* [4] A *posteriori* também pode ser usada para fazer previsões sobre eventos futuros.

[1] maiúsculo, pois se refere ao teorema de Bayes que é um sobrenome
[2] do inglês *prior distribution*
[3] do inglês *likelihood function*
[4] do inglês *posterior distribution*

O que muda da Estatística Frequentista?

- **Flexibilidade** - peças probabilísticas para construir um modelo[5]:
  - Conjecturas probabilísticas sobre os parâmetros:
    - *Priori*
    - Verossimilhança
- Melhor tratamento da **incerteza**:
  - Coerência
  - Propagação
  - Não se usa *"se amostrássemos infinitamente de uma população que não existe..."*
- Sem p**-valores**:
  - Todas as intuições estatísticas fazem **sentido**
  - 95% de certeza que o valor do parâmetro θ está entre x e y
  - Quase **impossível** fazer p-hacking.

[5] como se fosse LEGO

Um pouco mais de Formalidade

- Estatística Bayesiana usa declarações probabilísticas:
  - um ou mais parâmetros θ
  - dados não-observados $\tilde{y}$
- Essas declarações são condicionadas nos valores observados de y:
  - $P(\theta \mid y)$
  - $P(\tilde{y} \mid y)$
- Nós também, de maneira implícita, condicionados nos valores observados de quaisquer co-variáveis x

Principal Mudança

Definição (Estatística Bayesiana)

O uso do Teorema de Bayes[6] *como o procedimento de* ***estimativa dos parâmetros de interesse*** θ ***ou dados não-observados*** $\tilde{y}$. *(Gelman et al., 2013)*

[6] mais sobre ele já já...

Ferramentas para Estatística Bayesiana

- Stan
- PyMC
- JAGS
- BUGS

Stan[7]

- Plataforma para modelagem e computação estatística de alto desempenho
- Suporte financeiro da NUMFocus:
  - AWS Amazon
  - Bloomberg
  - Microsoft
  - IBM
  - RStudio
  - Facebook
  - NVIDIA
  - Netflix
- Linguagem própria, similar à C++
- Amostrador *Markov Chain Monte Carlo* (MCMC) em paralelo

[7] Carpenter et al. (2017)

Interfaces do Stan[8]

- R: RStan e CmdStanR
- Python: PyStan e CmdStanPy
- Shell (Linha de Comando): CmdStan
- Julia: Stan.jl
- Scala: ScalaStan
- ~~Matlab~~: ~~MatlabStan~~
- ~~Stata~~: ~~StataStan~~
- ~~Mathematica~~: ~~MathematicaStan~~

[8] Carpenter et al. (2017)

Interfaces Amigáveis do `Stan`[10]

Somente para R

- `rstanarm` (Goodrich et al., 2020): ajuda o usuário a especificar modelos usando a sintaxe familiar de fórmulas do R.
- `brms` (Bürkner, 2017): similar ao `rstanarm` pois usa a sintaxe familiar de fórmulas do R, mas dá maior flexibilidade na especificação de modelos mais complexos[9].

[9] e geralmente a amostragem é um pouco mais rápida que o `rstanarm`

[10] Carpenter et al. (2017)

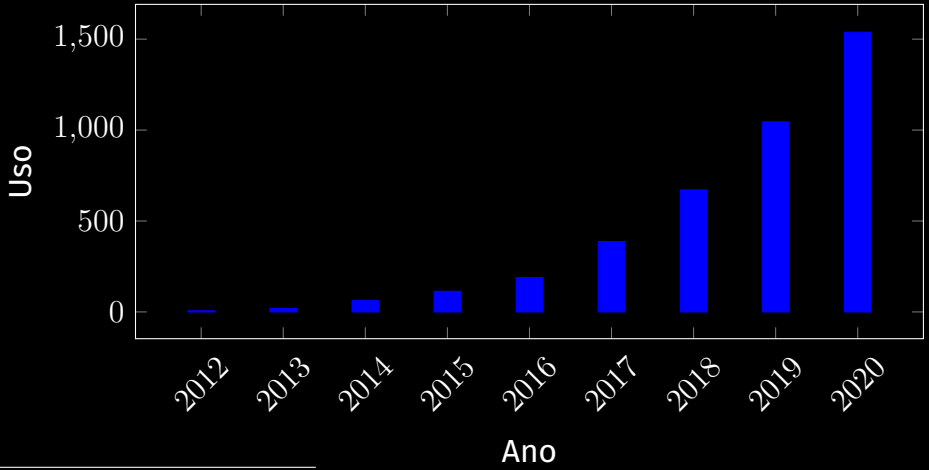
Código Stan versus Fórmulas de R

```
data {
  int<lower=0> N;
  vector[N] x1;
  vector[N] x2;
  vector[N] y;
}
parameters {
  real alpha;
  vector[2] beta;
  real<lower=0> sigma;
}
model {
  sigma ~ cauchy(0, 2.5);
  y ~ normal(alpha + beta[1] * x1 + beta[2] * x2, sigma);
}
```

Código Stan versus Fórmulas de R

```
stan_glm(y ~ x1 + x2, data = df, family = gaussian())
```

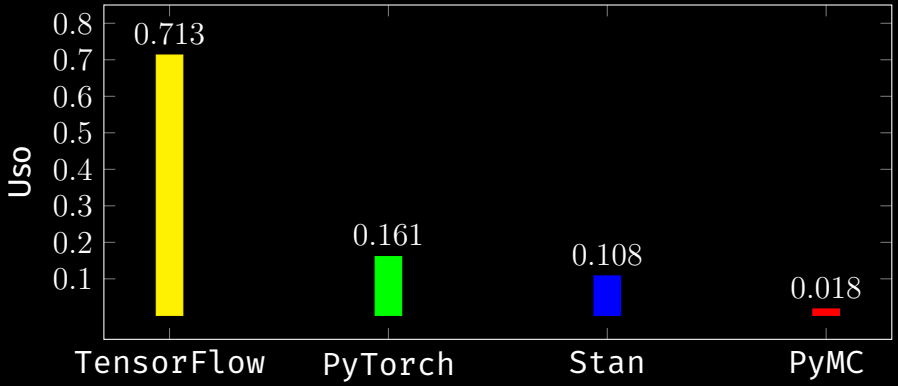
Stan[11] na Scopus[12]



[11] foi lançado em 2012

[12] veja as buscas Scopus nos Slides de Backup no final dessa apresentação

Stan[13][14]



[13] baseado no reporte anual do Breck Baldwin para a NUMFocus

[14] veja as buscas Scopus nos Slides de Backup no final dessa apresentação

R

- R é uma linguagem criada **por estatísticos para estatísticos**
- Possui um vasto **ecossistema de bibliotecas** e é amplamente usado na **ciência** e em especial nas **ciências aplicadas**
- Quase toda **tese maluca** ou **algoritmo inovador** de Estatística/Probabilidade está no CRAN (Repositório de Pacotes R
- Como linguagem de programação é **horrível**[15]: Recomendo `Julia`

[15] consegue ser um pouco menos pior que `Python`

Python e PyMC

- Python consegue ser um pouco melhor que R
- Mas tem a "tara" dos anos 90 de tudo ser **Orientado à Objetos**
- PyMC (Salvatier et al., 2016):
  - Uma Biblioteca de Estatística Bayesiana com o seu próprio amostrador *Markov Chain Monte Carlo* (MCMC)
  - Também com Suporte financeiro da NUMFocus
  - Amarram o cavalo num barco que afundou há algum tempo: Theano
  - Theano **morreu** mas os desenvolvedores do PyMC fizeram um *fork* no projeto e estão usando-o como *backend*

Exemplo de Código PyMC

```
with Model() as model:
    sigma = HalfCauchy("sigma", beta=10, testval=1.0)
    alpha = Normal("Intercept", 0, sigma=20)
    beta_1 = Normal("beta_1", 0, sigma=2)
    beta_2 = Normal("beta_2", 0, sigma=2)

    likelihood = Normal("y",
                        mu=alpha + beta_1 * x1 + beta_2 * x2, sigma=sigma, obs
```

Julia e Turing

- `Julia` (Bezanson et al., 2017) é uma linguagem relativamente nova, lançada pela primeira vez em 2012, que visa ser de **alto nível** e **rápida**
- Linguagem de tipagem **dinâmica rápida** que compila *just-in-time* (JIT) em código nativo usando `LLVM`.
- "Roda como C, mas lê como `Python`" (Perkel, 2019), o que significa que é extremamente **rápida**, fácil **prototipagem** e **ler/escrever** código.
- **Multi-paradigma**, combinando recursos de programação **imperativa**, **funcional** e **orientada a objetos**.

Turing

- `Turing` é uma **Linguagem de Programação Probabilística**[16] escrita totalmente em `Julia`
- Usa **pacotes** de `Julia` para:
  - **Diferenciação Automática**[17]
  - **Distribuições Probabilísticas**
  - **Solucionadores de Equações Ordinais**[18]
  - **Redes Neurais** (sendo responsável pela parte "Bayesiana" da Rede Neural Bayesiana)

[16] em inglês *probabilistic programming language* (PPL)
[17] *autodiff*
[18] *Ordinary Differential Equation Solvers* (ODE)

Exemplo de Código Turing[19]

```
@model linreg(x₁, x₂, y) = begin
    α ~ Normal(0, 20)
    β₁ ~ Normal(0, 2)
    β₂ ~ Normal(0, 2)
    σ ~ Exponential(1)

    y .~ Normal(α .+ β₁ * x₁ + β₂ * x₂, σ)
end
```

[19] eu acredito que há muito potencial em Julia e escrevi um tutorial de Estatística Bayesiana usando Julia e Turing (Storopoli, 2021a)

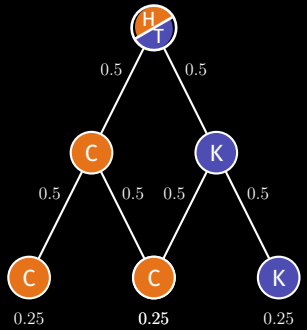
PROBABILIDADE NÃO EXISTE![20]

- Sim, a probabilidade não existe.
- Ou melhor, probabilidade como uma quantidade física, chance objetiva, **NÃO existe**
- se dispensarmos a questão da chance objetiva *nada se perde*
- A matemática do raciocínio indutivo permanece **exatamente a mesma**

[20] de Finetti (1974)

PROBABILIDADE NÃO EXISTE![21]

- Considere jogar uma moeda de enviesada
- As tentativas são consideradas independentes e, como resultado, exibem outra propriedade importante: **a ordem não importa**
- A frequência é considerada uma **estatística suficiente**
- Dizer que a ordem não importa ou dizer que a única coisa que importa é a frequência são duas maneiras de dizer exatamente a mesma coisa
- Dizemos que essa probabilidade é **invariante sob permutações**



[21] de Finetti (1974)

Interpretações da Probabilidade

- **Objetiva** - frequência no longo prazo de um evento específico
  - $P(\text{chuva}) = \frac{\text{dias que choveram}}{\text{dias totais}}$
  - $P(\text{chance de eu ser presidente} = 0)$ (Nunca ocorreu)

- **Subjetiva** - nível de crença em um evento
  - $P(\text{chuva}) =$ crença que choverá
  - $P(\text{chance de eu ser presidente} = 10^{-10})$ (Muito improvável)

O que é Probabilidade?

Definição (Probabilidade)

Sobre notação, definimos que A é um evento e $P(A)$ a probabilidade do evento, logo:

$$\{P(A) \in \mathbb{R} : 0 \leq P(A) \leq 1\}.$$

Isto quer dizer o "probabilidade do evento A ocorrer é o conjunto de todos os números reais entre 0 e 1; incluindo 0 e 1"

Axiomas da Probabilidade[22]

- **Não-negatividade**: Para todo A, $P(A) \geq 0$. Toda probabilidade é positiva (maior ou igual a zero), independente do evento
- **Aditividade**: Para dois *mutuamente exclusivos* A e B (não podem ocorrer ao mesmo tempo): $P(A) = 1 - P(B)$ e $P(B) = 1 - P(A)$
- **Normalização**: A probabilidade de todos os eventos possíveis $A_1, A_2, \ldots$ devem somar 1: $\sum_{n \in \mathbb{N}} A_n = 1$

[22] Kolmogorov (1933)

Probabilidade Condicional

Definição (Probabilidade Condicional)

Probabilidade de um evento ocorrer caso outro tenha ocorrido ou não.

A notação que usamos é $P(A \mid B)$, que lê-se como "a probabilidade de observamos A dado que já observamos B".

Probabilidade Conjunta

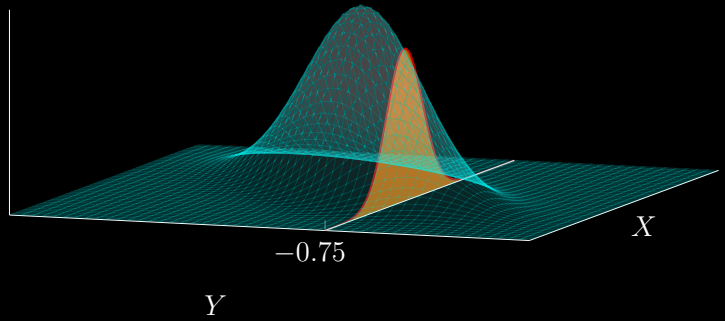
Definição (Probabilidade Conjunta)

Probabilidade de observados dois ou mais eventos ocorrem.

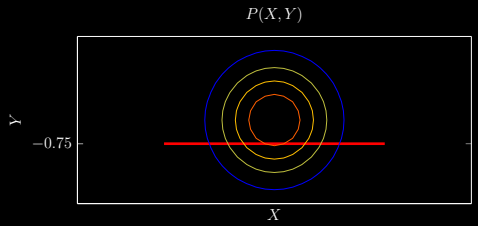
A notação que usamos é $P(A, B)$, que lê-se como "a probabilidade de observamos A e também observamos B".

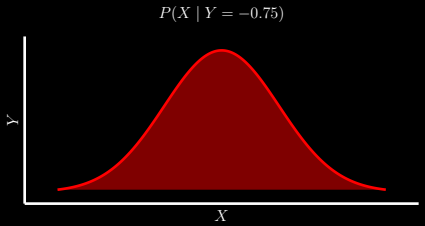
Exemplo de Probabilidade Conjunta vs Probabilidade Condicional

$P(X, Y)$ versus $P(X \mid Y = -0.75)$



Exemplo de Probabilidade Conjunta vs Probabilidade Condicional





Espaços Amostrais

- Discretos

$$\Theta = \{1, 2, \ldots, \}$$

- Contínuos

$$\Theta \in (-\infty, \infty)$$

Espaços Amostrais Discretos

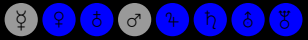
8 Planetas do Nosso Sistema Solar

- Mercúrio - ☿
- Vênus - ♀
- Terra - ♁
- Marte ♂
- Júpiter - ♃
- Saturno ♄
- Urano - ♅
- Netuno ♆

Espaços Amostrais Discretos

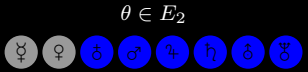
O planeta possui campo magnético

$\theta \in E_1$



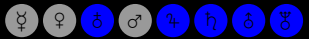
O planeta possui luas

$\theta \in E_2$



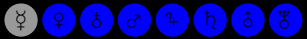
O planeta possui campo magnético e luas

$\theta \in E_1 \cap E_2$



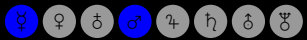
O planeta possui campo magnético ou luas

$\theta \in E_1 \cup E_2$



O planeta não possui um campo magnético

$\theta \in E_1^c$



Espaços Amostrais Contínuos

A distância é menos que cinco centímetros

$\theta \in E_1$

A distância é entre três e sete centímetros

$\theta \in E_2$

A distância é menos que cinco centímetros
e entre três e sete centímetros

$\theta \in E_1 \cap E_2$

A distância é menos que cinco centímetros
ou entre três e sete centímetros

$\theta \in E_1 \cup E_2$

A distância não é menos que cinco centímetros

$\theta \in E_1^c$

Parâmetros Discretos versus Contínuos

Tudo o que foi exposto até agora partiu do pressuposto que os parâmetros são discretos. Isto foi feito com o intuito de prover uma melhor intuição do que é probabilidade. Nem sempre trabalhamos com parâmetros discretos. Os parâmetros podem ser contínuos, como por exemplo: idade, altura, peso etc. Mas não se desespere, todas as regras e axiomas da probabilidade são válidos também para parâmetros contínuos. A única coisa que temos que fazer é trocar todas as somas $\sum$ por integrais $\int$. Por exemplo o terceiro axioma de **Normalização** para variáveis aleatórias contínuas se torna:

$$\int_{x \in X} p(x)dx = 1.$$

Sumário para Distribuições Probabilísticas

2.1 Leituras Recomendadas

2.2 Distribuições Discretas

2.2.1 Uniforme Discreta

2.2.2 Bernoulli

2.2.3 Binomial

2.2.4 Poisson

2.2.5 Binomial Negativa

2.3 Distribuições Contínuas

2.3.1 Uniforme Contínua

2.3.2 Normal

2.3.3 Log-Normal

2.3.4 Exponencial

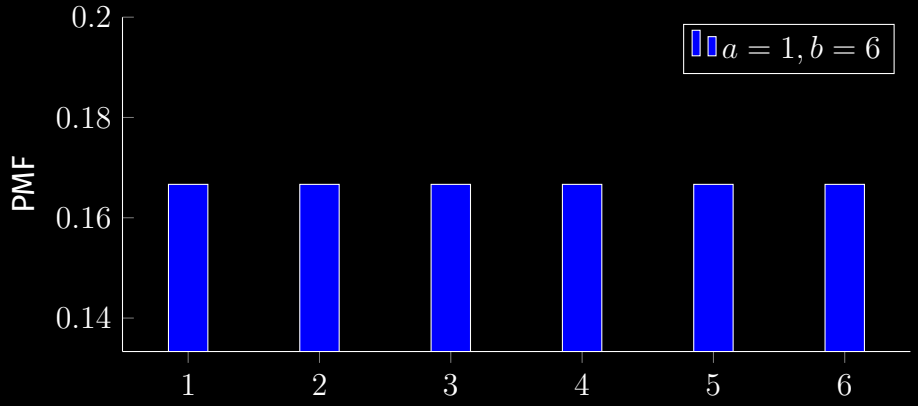
2.3.5 t de Student

2.3.6 Beta

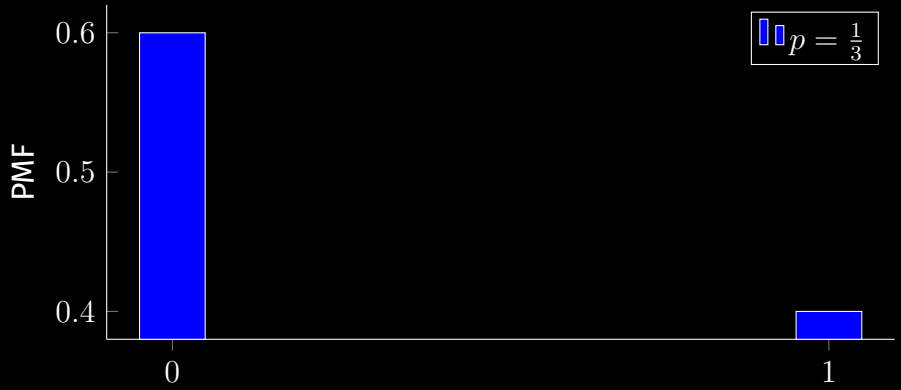
Distribuições Probabilísticas - Leituras Recomendadas

- Placeholder

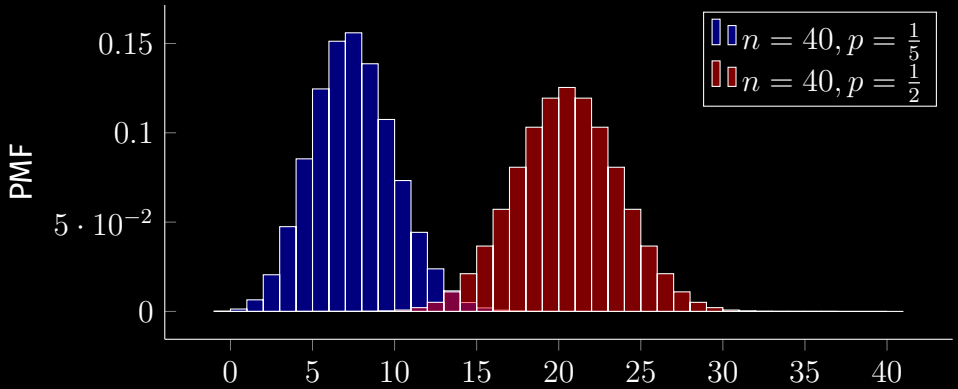
Uniforme Discreta



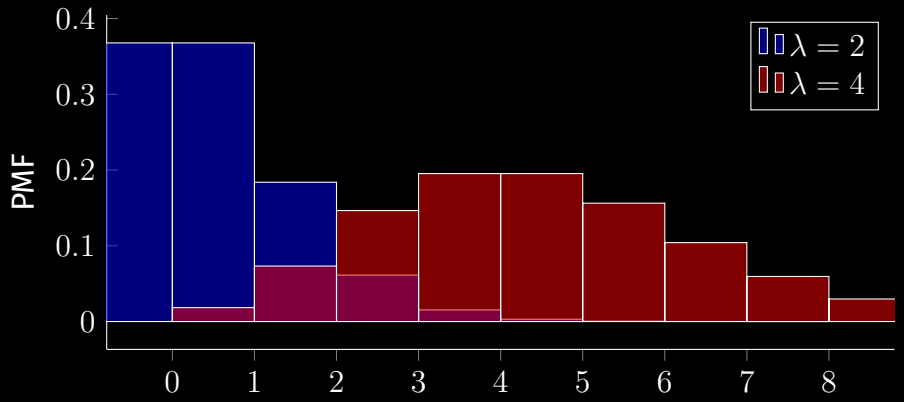
Bernoulli



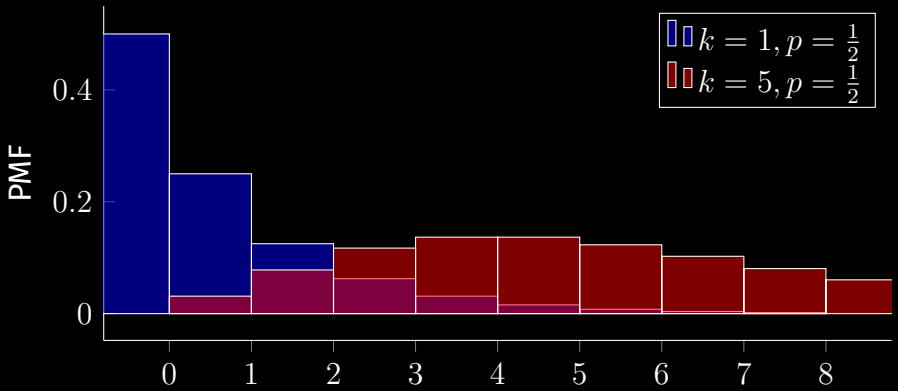
Binomial


PMF
0.15
0.1
$5 \cdot 10^{-2}$
0
0
5
10
15
20
25
30
35
40
$n = 40, p = \frac{1}{5}$
$n = 40, p = \frac{1}{2}$

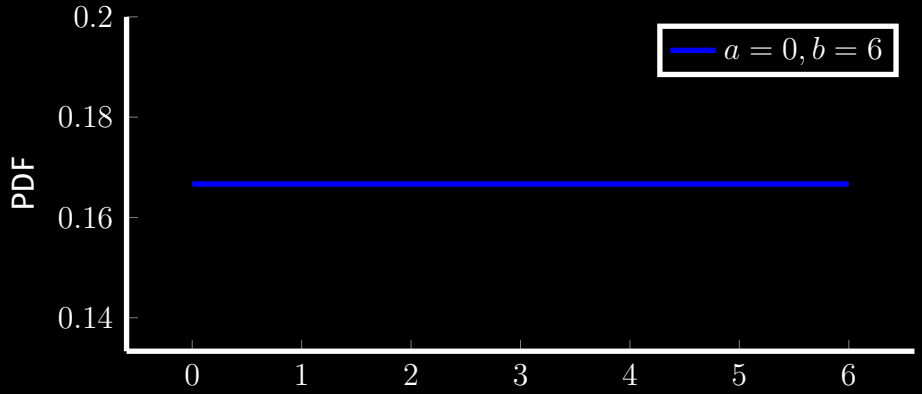
Poisson



Binomial Negativa



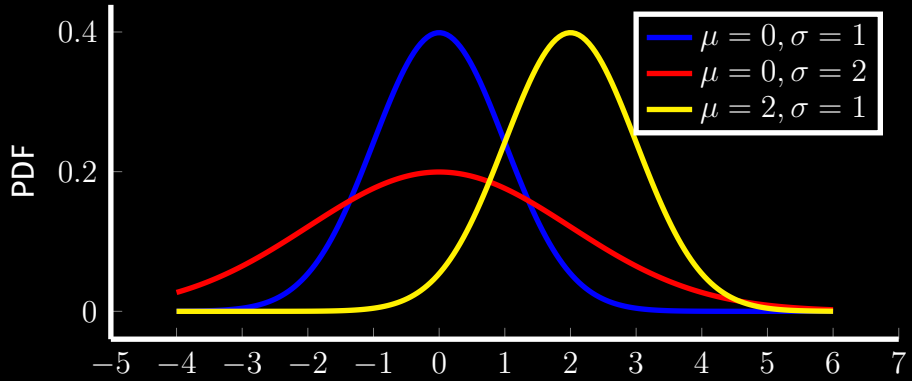
Uniforme Contínua



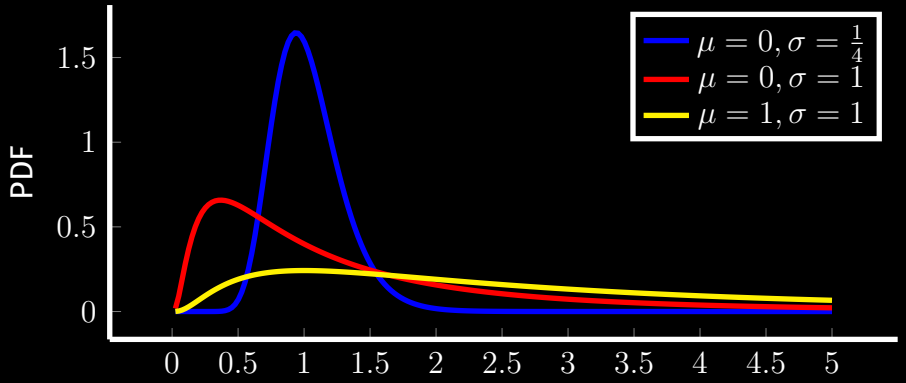
Normal[23]

[23] Veja como a distribuição Normal foi derivada a partir da distribuição binomial nos Slides de Backup no final dessa apresentação

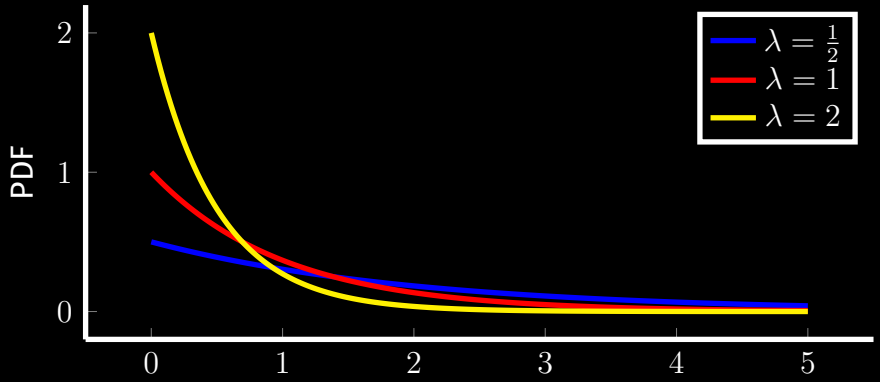
Normal



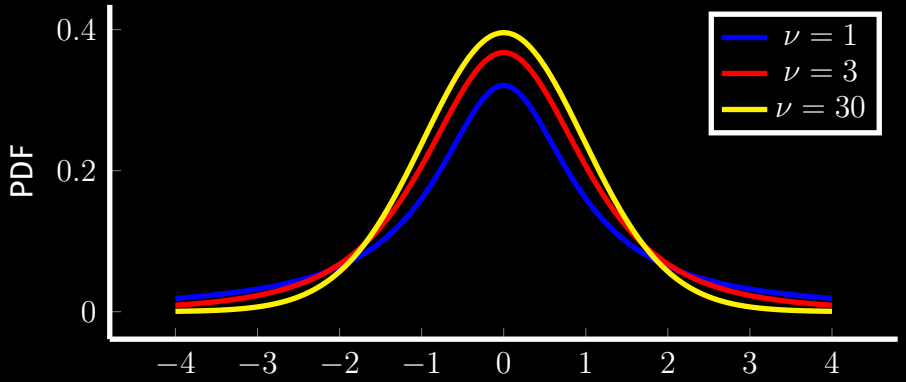
Log-Normal


PDF
1.5
1
0.5
0
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5
$\mu = 0, \sigma = \frac{1}{4}$
$\mu = 0, \sigma = 1$
$\mu = 1, \sigma = 1$

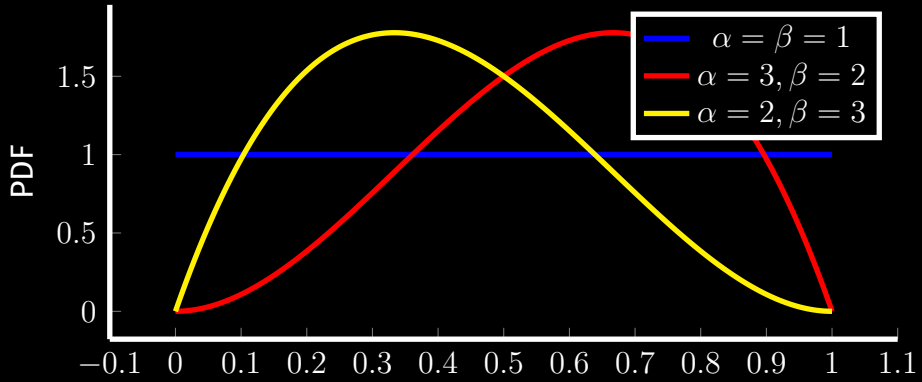
Exponencial



t de Student



Beta


PDF
1.5
1
0.5
0
−0.1
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
1.1
$\alpha = \beta = 1$
$\alpha = 3, \beta = 2$
$\alpha = 2, \beta = 3$

Sumário para `rstanarm` e `brms`

3.1 Leituras Recomendadas

`rstanarm` e `brms` - Leituras Recomendadas

- Placeholder

Sumário para Priors

4.1 Leituras Recomendadas

Priors - Leituras Recomendadas

- Gelman et al. (2013):
  - Capítulo 2: Single-parameter models
  - Capítulo 3: Introduction to multiparameter models

- McElreath (2020) - Capítulo 4: Geocentric Models

- van de Schoot et al. (2021)

- Storopoli (2021b) - Priors

Sumário para Markov Chain Monte Carlo - MCMC

5.1 Leituras Recomendadas
5.2 Por quê Precisamos de MCMC?
5.3 Algoritmos de MCMC
5.3.1 Metropolis
5.3.2 Metropolis-Hastings
5.3.3 As Limitações de Metropolis e Metropolis-Hastings
5.3.4 Gibbs
5.3.5 As Limitações de Gibbs
5.3.6 Hamiltonian Monte Carlo (HMC)
5.3.7 No-U-Turn-Sampler (NUTS)
5.3.8 As Limitações de HMC e NUTS
5.4 Convergência de Correntes Markov
5.4.1 Métricas de Convergência
5.4.2 Visualizações de Convergência
5.4.3 O que fazer se Correntes Markov não Convergirem

Markov Chain Monte Carlo - MCMC - Leituras Recomendadas

- Gelman et al. (2013)
  - Capítulo 10: Introduction to Bayesian computation
  - Capítulo 11: Basics of Markov chain simulation
  - Capítulo 12: Computationally efficient Markov chain simulation
- McElreath (2020) - Capítulo 9: Markov Chain Monte Carlo
- Neal (2011)
- Betancourt (2017)
- Gelman et al. (2020) - Capítulo 22, Seção 22.8: Computational efficiency
- Storopoli (2021b) - Markov Chain Monte Carlo
- Chib e Greenberg (1995)
- Casella e George (1992)

Algoritmo de Metropolis[24]

Algoritmo 1: Metropolis

Defina um ponto inicial $\boldsymbol{\theta}^{(0)} \in \mathbb{R}^p$ do qual $P\left(\boldsymbol{\theta}^{(0)} \mid \boldsymbol{y}\right) > 0$

para $t = 1, 2, \ldots$

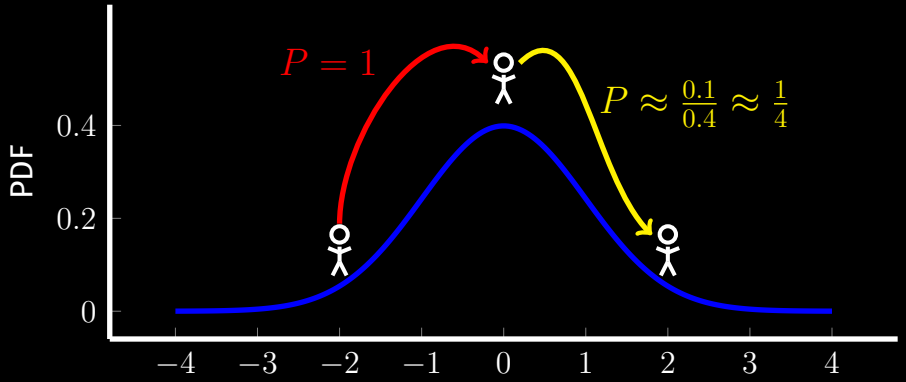
Amostra uma proposta $\boldsymbol{\theta}^{(*)}$ de uma distribuição de propostas no tempo t, $J_t\left(\boldsymbol{\theta}^{(*)} \mid \boldsymbol{\theta}^{(t-1)}\right)$

Como regra de aceitação/rejeição calcule a proporção das probabilidades: $r = \frac{P\left(\boldsymbol{\theta}^{(*)} \mid \boldsymbol{y}\right)}{P\left(\boldsymbol{\theta}^{(t-1)} \mid \boldsymbol{y}\right)}$

Designe: $\boldsymbol{\theta}^{(t)} = \begin{cases} \boldsymbol{\theta}^{(*)} & \text{com probabilidade } \min(r, 1) \\ \boldsymbol{\theta}^{(t-1)} & \text{caso contrário} \end{cases}$

[24] Metropolis et al. (1953)

Intuição Visual de Metropolis



Algoritmo de Metropolis - Hastings[25]

Veja Metropolis em ação no `chi-feng/mcmc-demo`

Algoritmo 2: Metropolis-Hastings

Defina um ponto inicial $\boldsymbol{\theta}^{(0)} \in \mathbb{R}^p$ do qual $P\left(\boldsymbol{\theta}^{(0)} \mid \boldsymbol{y}\right) > 0$

para $t = 1, 2, \ldots$

Amostra uma proposta $\boldsymbol{\theta}^{(*)}$ de uma distribuição de propostas no tempo t, $J_t\left(\boldsymbol{\theta}^{(*)} \mid \boldsymbol{\theta}^{(t-1)}\right)$

Como regra de aceitação/rejeição calcule a proporção das probabilidades: $r = \dfrac{\frac{P\left(\boldsymbol{\theta}^{(*)} \mid \boldsymbol{y}\right)}{J_t\left(\boldsymbol{\theta}^{(*)} \mid \boldsymbol{\theta}^{(t-1)}\right)}}{\frac{P\left(\boldsymbol{\theta}^{(t-1)} \mid \boldsymbol{y}\right)}{J_t\left(\boldsymbol{\theta}^{(t-1)} \mid \boldsymbol{\theta}^{(*)}\right)}}$

Designe: $\boldsymbol{\theta}^{(t)} = \begin{cases} \boldsymbol{\theta}^{(*)} & \text{com probabilidade } \min(r, 1) \\ \boldsymbol{\theta}^{(t-1)} & \text{caso contrário} \end{cases}$

[25] Hastings (1970)

Algoritmo de Gibbs[26]

Veja Gibbs em ação no `chi-feng/mcmc-demo`

Algoritmo 3: Gibbs

Defina um ponto inicial $\boldsymbol{\theta}^{(0)} \in \mathbb{R}^p$ do qual $P\left(\boldsymbol{\theta}^{(0)} \mid \boldsymbol{y}\right) > 0$

para $t = 1, 2, \ldots$

Designe: $\boldsymbol{\theta}^{(t)} = \begin{cases} \theta_1^{(t)} & \sim P\left(\theta_1 \mid \theta_2^{(0)}, \ldots, \theta_p^{(0)}\right) \\ \theta_2^{(t)} & \sim P\left(\theta_2 \mid \theta_1^{(t-1)}, \ldots, \theta_p^{(0)}\right) \\ & \vdots \\ \theta_p^{(t)} & \sim P\left(\theta_p \mid \theta_1^{(t-1)}, \ldots, \theta_{p-1}^{(t-1)}\right) \end{cases}$

[26] Geman e Geman (1984)

Algoritmo de HMC[27]

Veja HMC em ação no `chi-feng/mcmc-demo`

Algoritmo 4: Hamiltonian Monte Carlo (HMC)

Defina um ponto inicial $\boldsymbol{\theta}^{(0)} \in \mathbb{R}^p$ do qual $P\left(\boldsymbol{\theta}^{(0)} \mid \boldsymbol{y}\right) > 0$

Amostre $\boldsymbol{\phi}$ de uma Normal$(\mathbf{0}, \mathbf{M})$

Simultaneamente amostre $\boldsymbol{\theta}$ e $\boldsymbol{\phi}$ com L *leapfrog steps* cada um reduzido por um fator ϵ.

para $1, 2, \ldots, L$

> Use o gradiente do $\log$ da posterior de $\boldsymbol{\theta}$ para produzir um *half-step* de $\boldsymbol{\phi}$: $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \frac{1}{2}\epsilon \frac{d \log P(\boldsymbol{\theta}|\boldsymbol{y})}{d\theta}$
>
> Use $\boldsymbol{\phi}$ para atualizar $\boldsymbol{\theta}$: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \mathbf{M}^{-1}\boldsymbol{\phi}$
>
> Novamente use o gradiente de $\boldsymbol{\theta}$ para produzir um *half-step* de $\boldsymbol{\phi}$: $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \frac{1}{2}\epsilon \frac{d \log P(\boldsymbol{\theta}|\boldsymbol{y})}{d\theta}$

Como regra de aceitação/rejeição calcule: $r = \frac{P\left(\boldsymbol{\theta}^{(*)}|\boldsymbol{y}\right)P\left(\boldsymbol{\phi}^{(*)}\right)}{P\left(\boldsymbol{\theta}^{(t-1)}|\boldsymbol{y}\right)P\left(\boldsymbol{\phi}^{(t-1)}\right)}$

Designe: $\boldsymbol{\theta}^{(t)} = \begin{cases} \boldsymbol{\theta}^{(*)} & \text{com probabilidade } \min(r, 1) \\ \boldsymbol{\theta}^{(t-1)} & \text{caso contrário} \end{cases}$

[27] Neal (2011)

Algoritmo de NUTS[28]

Veja NUTS em ação no `chi-feng/mcmc-demo`

Algoritmo 5: No-U-Turn-Sampler (NUTS)

[28] Hoffman e Gelman (2011)

Sumário para Regressão Linear Bayesiana

6.1 Leituras Recomendadas

Regressão Linear Bayesiana - Leituras Recomendadas

- Placeholder

Sumário para Regressão Logística Bayesiana

7.1 Leituras Recomendadas

Regressão Logística Bayesiana - Leituras Recomendadas

- Placeholder

Sumário para Regressão de Poisson Bayesiana

8.1 Leituras Recomendadas

Regressão de Poisson Bayesiana - Leituras Recomendadas

- Placeholder

Sumário para Regressão Robusta Bayesiana

9.1 Leituras Recomendadas

Regressão Robusta Bayesiana - Leituras Recomendadas

- Placeholder

10.1 Leituras Recomendadas

Modelos Multiníveis Bayesianos - Leituras Recomendadas

- Placeholder

Sumário para Comparação de Modelos

11.1 Leituras Recomendadas

Comparação de Modelos - Leituras Recomendadas

- Placeholder

Referências I

Betancourt, M. (2017, janeiro 9). *A Conceptual Introduction to Hamiltonian Monte Carlo*. arXiv: 1701.02434. Obtido 6 novembro 2019, de http://arxiv.org/abs/1701.02434

Bezanson, J., Edelman, A., Karpinski, S. & Shah, V. B. (2017). Julia: A Fresh Approach to Numerical Computing. *SIAM review*, *59*(1), 65–98.

Bürkner, P.-C. (2017). brms: An R Package for Bayesian Multilevel Models Using Stan. *Journal of Statistical Software*, *80*(1), 1–28. https://doi.org/10.18637/jss.v080.i01

Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P. & Riddell, A. (2017). Stan : A Probabilistic Programming Language. *Journal of Statistical Software*, *76*(1). https://doi.org/10.18637/jss.v076.i01

Referências II

Casella, G. & George, E. (1992). Explaining the Gibbs Sampler [Publisher: Taylor & Francis _eprint: https://www.tandfonline.com/doi/pdf/10.1080/00031305.1992.10475878]. *The American Statistician*, *46*(3), 167–174. https://doi.org/10.1080/00031305.1992.10475878

Chib, S. & Greenberg, E. (1995). Understanding the Metropolis-Hastings Algorithm. *The American Statistician*, *49*(4), 327–335. https://doi.org/10.1080/00031305.1995.10476177

de Finetti, B. (1974). *Theory of Probability* (Volume 1). John Wiley & Sons.

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A. & Rubin, D. B. (2013). *Bayesian Data Analysis*. Chapman and Hall/CRC.

Gelman, A., Hill, J. & Vehtari, A. (2020). *Regression and Other Stories*. Cambridge University Press.

Referências III

Geman, S. & Geman, D. (1984). Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *PAMI-6*(6), 721–741. https://doi.org/10.1109/TPAMI.1984.4767596

Goodrich, B., Gabry, J., Ali, I. & Brilleman, S. (2020). rstanarm: Bayesian applied regression modeling via Stan. [R package version 2.21.1]. https://mc-stan.org/rstanarm

Hastings, W. K. (1970). Monte Carlo Sampling Methods Using Markov Chains and Their Applications. *Biometrika*, *57*(1), 97–109. https://doi.org/10.1093/biomet/57.1.97

Referências IV

Hoffman, M. D. & Gelman, A. (2011). The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, *15*(1), 1593–1623. http://arxiv.org/abs/1111.4246

Kolmogorov, A. N. (1933). *Foundations of the Theory of Probability*. Julius Springer.

McElreath, R. (2020). *Statistical Rethinking: A Bayesian Course with Examples in R and Stan*. CRC press.

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. & Teller, E. (1953). Equation of State Calculations by Fast Computing Machines. *The Journal of Chemical Physics*, *21*(6), 1087–1092. https://doi.org/10.1063/1.1699114

Referências V

Neal, R. M. (2011). MCMC Using Hamiltonian Dynamics. Em S. Brooks, A. Gelman, G. L. Jones & X.-L. Meng (Eds.), *Handbook of Markov Chain Monte Carlo*.

Perkel, J. M. (2019). Julia: Come for the Syntax, Stay for the Speed. *Nature*, *572*(7767), 141–142. https://doi.org/10.1038/d41586-019-02310-3

Salvatier, J., Wiecki, T. V. & Fonnesbeck, C. (2016). Probabilistic programming in Python using PyMC3. *PeerJ Computer Science*, *2*, e55.

Storopoli, J. (2021a). Bayesian Statistics with Julia and Turing. https://storopoli.io/Bayesian-Julia

Storopoli, J. (2021b). Estatística Bayesiana com R e Stan. https://storopoli.io/Estatistica-Bayesiana

Referências VI

van de Schoot, R., Depaoli, S., King, R., Kramer, B., Märtens, K., Tadesse, M. G., Vannucci, M., Gelman, A., Veen, D., Willemsen, J. & Yau, C. (2021). Bayesian statistics and modelling. *Nature Reviews Methods Primers*, *1*(1), 1–26. https://doi.org/10.1038/s43586-020-00001-2

Como citar esse Conteúdo

Storopoli, J. (2021b). Estatística Bayesiana com R e Stan. https://storopoli.io/Estatistica-Bayesiana

Buscas Scopus do Stan

- Uso Geral na Scopus: `ALL((brms AND burkner) OR (gelman AND hoffman AND stan) OR mc-stan.org OR rstanarm OR pystan OR (rstan AND NOT mit))`

- Uso em Comparação com outras Ferramentas Bayesianas: `REF((gelman AND hoffman AND stan) OR mc-stan.org) AND REF(brms AND burkner) AND REF(pystan) AND REF(rstanarm) AND REF(rstan AND NOT mit)`

- Uso das Ferramentas Bayesians: a busca anterior adicionado de `REF(PyMC3 OR (PyMC* AND fonnesbeck)) AND REF(tensorflow) AND REF(pytorch) AND REF(Keras)`

Como surgiu a distribuição Normal